

Caderno de Desafios Rápidos

Aula 02 — Herança, Polimorfismo, Composição e Classes Abstratas

Este caderno utiliza duas técnicas científicas de aprendizagem: Prática de Recuperação (recuperar ativamente é mais eficaz do que reler) e Repetição Espaçada (revisitar conceitos anteriores fortalece a memória de longo prazo). Tente responder cada desafio sem consultar o material — depois confira o gabarito no final.

■ **Tempo sugerido: 25-30 minutos** | ■ **Formato: individual ou em duplas** | ■ **Gabarito: páginas finais**

SEÇÃO 1 — REVISÃO ESPAÇADA | Conceitos da Aula 01 | 5 minutos

Recupere o que você aprendeu na aula anterior!

Desafio 1 [Revisão]

Explique a diferença entre CLASSE, OBJETO e INSTANCIACÃO. Use o exemplo de Carro nas três definições.

Dica: Aula 01 — planta x casa x construir.

Desafio 2 [Revisão]

Escreva um construtor para a classe Aluno que recebe nome (String) e matricula (int) e inicializa os atributos correspondentes.

Dica: Lembre-se: o construtor tem o mesmo nome da classe e usa this.

Desafio 3 [Revisão]

Por que declaramos atributos como private? E qual o papel dos getters e setters nesse contexto?

Dica: Encapsulamento e controle de acesso ao estado interno.

Desafio 4 [Revisão]

Identifique o erro no trecho abaixo e explique como corrigir:

```
public class Conta {  
    private double saldo;  
}  
// no main:  
Conta c = new Conta();  
c.saldo = 1000;
```

Dica: private bloqueia acesso direto. Como expor a alteração?

SEÇÃO 2 — CONTEÚDO DA AULA: BÁSICO | Herança e composição | 10 minutos

Teste o que você acabou de aprender!

Desafio 5 [Básico]

Verdadeiro ou Falso? Justifique cada resposta:

- a) Em Java, uma classe pode herdar de várias superclasses ao mesmo tempo.
- b) A relação 'Cachorro é-um Animal' é modelada por herança.
- c) A relação 'Carro tem-um Motor' é modelada por composição.
- d) Subclasses só podem usar atributos da superclasse se estes forem public.

Dica: Lembre-se de IS-A, HAS-A e dos modificadores de acesso.

Desafio 6 [Básico]

Crie a hierarquia: classe Animal com atributo nome e método respirar(); classe Cachorro que herda de Animal e adiciona o método latir(). Mostre o código completo das duas classes.

Dica: Use extends para criar a relação de herança.

Desafio 7 [Básico]

Olhe o código abaixo. Identifique se a relação modelada é herança ou composição e justifique a escolha:

```
public class Notebook {  
    private Teclado teclado = new Teclado();  
    private Tela tela = new Tela();  
}
```

Dica: Notebook É-UM Teclado? Ou Notebook TEM-UM Teclado?

Desafio 8 [Básico]

O código abaixo não compila. Identifique o motivo e proponha a correção:

```
public class Veiculo {  
    public Veiculo(String modelo) { /* ... */ }  
}  
public class Carro extends Veiculo {  
    public Carro() { /* construtor da subclasse */ }  
}
```

Dica: O que acontece quando a superclasse não tem construtor sem argumentos?

SEÇÃO 3 — CONTEÚDO DA AULA: INTERMEDIÁRIO | Polimorfismo e classes abstratas

Desafie seu raciocínio!

Desafio 9 [Intermediário]

Considere a classe abstrata `FormaGeometrica` com o método abstrato `calcularArea()`. Crie duas subclasses concretas — `Retangulo` (atributos `largura` e `altura`) e `Circulo` (atributo `raio`) — implementando o cálculo correto da área em cada uma.

Dica: Use `super()` no construtor e `@Override` no método.

Desafio 10 [Intermediário]

O que o código abaixo imprime? Justifique passo a passo:

```
public class Animal {
    public void falar() { System.out.println("som generico"); }
}
public class Gato extends Animal {
    @Override
    public void falar() { System.out.println("miau"); }
}
// main:
Animal a = new Gato();
a.falar();
```

Dica: Qual tipo decide a versão executada: o declarado ou o real?

Desafio 11 [Intermediário]

Você precisa modelar um sistema de pagamentos. Cada tipo de pagamento (`Dinheiro`, `Cartao`, `Pix`) tem um método `pagar(double valor)` com lógica própria, mas todos compartilham o atributo `descricao`. Decida: classe abstrata ou apenas interface? Justifique e implemente a estrutura básica.

Dica: Há atributo compartilhado entre os tipos? Existe código que vai ser usado por todas as subclasses?

Desafio 12 [Intermediário]

Considere os trechos abaixo. Em qual deles ocorre um problema em tempo de execução? Identifique e explique:

```
// (a)
Animal a = new Cachorro();
Cachorro c = (Cachorro) a;
c.latir();

// (b)
Animal a = new Gato();
Cachorro c = (Cachorro) a;
c.latir();
```

Dica: Downcasting requer que o objeto real seja realmente daquele tipo.

GABARITO

Errar é parte do processo! O objetivo do caderno é identificar onde você precisa revisar. Compare suas respostas com as soluções a seguir.

Resposta 1

CLASSE é a definição abstrata — descreve atributos e métodos que todo carro terá (ex: modelo, ano, ligar). OBJETO é uma instância concreta dessa classe — o Fusca azul de placa ABC-1234 que existe na memória. INSTANCIACÃO é o ato de pedir ao Java a criação do objeto: Carro c = new Carro();. Planta x casa concreta x construção.

Resposta 2

```
public class Aluno {
    private String nome;
    private int    matricula;

    public Aluno(String nome, int matricula) {
        this.nome      = nome;
        this.matricula = matricula;
    }
}
```

Resposta 3

Atributos private impedem que outras classes leiam ou alterem o estado diretamente. Getters expõem o valor de forma controlada (apenas leitura). Setters expõem a alteração de forma controlada, permitindo validar o valor antes de aceitá-lo. Essa combinação é o encapsulamento — protege a integridade do objeto e permite trocar a implementação interna sem afetar quem usa.

Resposta 4

Erro: o atributo saldo é private, então não pode ser acessado fora da classe Conta. Correção: expor um método público (setter ou operação de negócio):

```
public class Conta {
    private double saldo;
    public void depositar(double valor) {
        if (valor > 0) saldo += valor;
    }
    public double getSaldo() { return saldo; }
}
// no main:
Conta c = new Conta();
c.depositar(1000);
```

Resposta 5

- a) FALSO — Java permite herança simples de classes; para múltipla, usa interfaces.
- b) VERDADEIRO — IS-A é a marca registrada da herança.
- c) VERDADEIRO — HAS-A indica composição (um objeto contém outro como atributo).
- d) FALSO — subclasses também acessam membros protected. Apenas private bloqueia o acesso, mesmo para subclasses.

Resposta 6

```
public class Animal {
```

```
protected String nome;
public void respirar() {
    System.out.println(nome + " esta respirando.");
}

public class Cachorro extends Animal {
    public void latir() {
        System.out.println(nome + " disse Au!");
    }
}
```

Resposta 7

COMPOSIÇÃO. Notebook não é um Teclado nem uma Tela — ele TEM um teclado e TEM uma tela como partes internas. A relação IS-A não se sustenta (Notebook é um Teclado? Não!), enquanto HAS-A é natural. Composição é a escolha correta sempre que pensar em 'é parte de' ou 'tem como peça'.

Resposta 8

Erro: a superclasse Veiculo só tem um construtor que exige o argumento modelo. O construtor de Carro não chama super(...) explicitamente, então o Java tenta inserir super() sem argumentos — que não existe — e o código não compila. Correções possíveis:

```
// Opcao 1: chamar super com argumento
public Carro() {
    super("Modelo padrao");
}

// Opcao 2: adicionar construtor sem argumentos em Veiculo
public Veiculo() { /* ... */ }
```

Resposta 9

```
public abstract class FormaGeometrica {
    private String cor;
    public FormaGeometrica(String cor) { this.cor = cor; }
    public abstract double calcularArea();
}

public class Retangulo extends FormaGeometrica {
    private double largura, altura;
    public Retangulo(String cor, double l, double a) {
        super(cor);
        this.largura = l;
        this.altura = a;
    }
    @Override
    public double calcularArea() { return largura * altura; }
}

public class Circulo extends FormaGeometrica {
    private double raio;
    public Circulo(String cor, double raio) {
        super(cor);
        this.raio = raio;
    }
    @Override
    public double calcularArea() { return Math.PI * raio * raio; }
}
```

Resposta 10

Saída: miau

Justificativa: o tipo DECLARADO da variável é Animal, mas o tipo REAL do objeto em memória é Gato. O Java executa a versão sobrescrita em Gato (despacho dinâmico). Esse é o coração do polimorfismo.

Resposta 11

CLASSE ABSTRATA é a escolha mais natural aqui, porque há ATRIBUTO comum (descricao) entre todos os tipos de pagamento — e classes abstratas podem carregar estado, enquanto interfaces não. Estrutura básica:

```
public abstract class Pagamento {
```

```
    protected String descricao;  
    public Pagamento(String descricao) { this.descricao = descricao; }  
    public abstract void pagar(double valor);  
}  
  
public class Pix extends Pagamento {  
    public Pix(String d) { super(d); }  
    @Override  
    public void pagar(double valor) {  
        System.out.println("Pix de R$ " + valor);  
    }  
}
```

Se NÃO houvesse estado compartilhado, uma interface seria a melhor escolha. Como veremos na Aula 03, interfaces brilham quando o foco é apenas o contrato.

Resposta 12

O trecho (b) lança `ClassCastException` em tempo de execução. Justificativa: o tipo real do objeto é `Gato`, mas o cast tenta forçá-lo para `Cachorro`. Como `Gato` não é uma subclasse de `Cachorro`, o Java aborta. Em (a), o objeto real É um `Cachorro`, então o downcasting é válido. Para evitar esse erro, use `instanceof` antes do cast: `if (a instanceof Cachorro) { ... }`

SENAC Entra21 — Módulo 07: Programação Orientada a Objetos — Material de apoio ao aluno